



Sprout — All Prompts & Responses (Full)

From the very first “build the app” command · in order · English

Digital lending platform

Prompts 1-15 are reconstructed from the git commit history. The original chat that first created the app was compacted and isn't recoverable word-for-word, so these are the commands that produced each committed step (paraphrased), with a one-line note on what was delivered. From Prompt 16 onward, the prompts and responses are the real, verbatim conversation.

PROMPT 1 · RECONSTRUCTED

“Build me a digital loan-origination app called Sprout — a customer app with the full set of screens, packaged so it can run in a browser / as a PWA.”

DELIVERED

Created Sprout LOS: a 32-screen design gallery + a WebView/PWA customer app.

PROMPT 2 · RECONSTRUCTED

“Put it online so I can preview it live.”

DELIVERED

Added a GitHub Pages workflow so every push gets a live preview link.

PROMPT 3 · RECONSTRUCTED

“Make everything English but keep the ₱ currency.”

DELIVERED

Localized all screens to English while keeping ₱, and added self-contained single-file builds.

PROMPT 4 · RECONSTRUCTED

“Fit the app to a phone screen (iPhone frame on desktop) and lock scrolling so only the content scrolls inside the frame; tighten the spacing and add a modern bottom nav.”

DELIVERED

Locked the shell to iPhone 17 Pro (402×874) with a titanium bezel on desktop, fixed the scroll, and added an SVG bottom nav with an active pill.

PROMPT 5 · RECONSTRUCTED

“Redesign Home like a mobile-banking app — an immersive gradient header (no blank top area), a bold balance, and circular quick actions with an activity feed.”

DELIVERED

Rebuilt Home as a mobile-banking layout with an immersive gradient header (hidden top bar from first paint), bold balance, circular quick actions, and an activity feed.

PROMPT 6 • RECONSTRUCTED

“Build the full customer journey flow (onboarding → create account → verify → login → apply), ready for both store and browser.”

DELIVERED

Built the full customer-journey app flow (web + store ready) and bumped the service-worker cache so the full flow replaced the stale one.

PROMPT 7 • RECONSTRUCTED

“Rebuild the auth screens (onboarding/create/verify/login) to match the design, rewrite the borrowing/estimate screen in plain language (drop the internal jargon boxes), and make every loan product selectable.”

DELIVERED

Rebuilt auth with full button/field tests, rewrote the estimate screen in plain language, simplified the loan request and made every product selectable with docked CTAs.

PROMPT 8 • RECONSTRUCTED

“Add a back office backed by Supabase to manage the product catalogue, align the Home progress, and make sure customers never hit a dead-end screen.”

DELIVERED

Added a Supabase-backed product catalogue + admin UI, aligned application progress on Home, and guaranteed no dead-end screens for customers.

PROMPT 9 • RECONSTRUCTED

“Add real e-KYC: capture the ID card (front + back) with the camera and a photo review; make the occupation field a searchable typeahead and the pay-type a working dropdown.”

DELIVERED

Added real camera capture (front+back) with photo review and a pluggable ID-verify hook, plus a searchable occupation typeahead and a working pay-type dropdown.

PROMPT 10 • RECONSTRUCTED

“Make the document-upload step functional — camera, photos or files — with a preview.”

DELIVERED

Shipped a functional upload step (camera/photos/files) + preview, and eliminated stale-cache issues.

PROMPT 11 • RECONSTRUCTED

“Add back-office Pre-screening Rules that actually drive customer screening, plus Document requirements (required/optional), a FATCA answer rule, and an Approved-vs-Under-review evaluation.”

DELIVERED

Added rule-driven Pre-screening + Document requirements, made FATCA a choose-the-passing-answer rule, and implemented Approved-vs-Under-review evaluation by the rules.

PROMPT 12 • RECONSTRUCTED

“Build the officer app (/staff) with maker-checker case review, and let the customer see the real officer decision.”

DELIVERED

Built the officer app (/staff) with maker-checker review and integrated the real decision back to the customer.

PROMPT 13 • RECONSTRUCTED

“Add role-aware login that routes staff vs customer from one entry, and replace the mock login with real customer authentication.”

DELIVERED

Added role-aware login (staff vs customer from one entry) and real customer authentication (no more mock login).

PROMPT 14 • RECONSTRUCTED

“Add back-office Staff/role management with per-case segregation of duties, and add officer/manager roles so officers can escalate special cases to a manager.”

DELIVERED

Added Staff/role management + per-case segregation of duties, and officer/manager roles with escalation of special cases to a manager (manager only decides escalated cases).

PROMPT 15 • RECONSTRUCTED

“Add real document upload with an officer view + status, and fix the 400 error customers hit when uploading documents.”

DELIVERED

Added real document upload with officer view & status, surfaced upload errors with inline image preview, and fixed the 400 (storage RLS role mismatch).

PROMPT 16

“Why isn’t it done yet?”

RESPONSE

Done and pushed (deploy running). Here’s the honest summary of what actually happened:

Why officers still don’t see the files

It’s not a bug in the current code — the DB side is fixed and live; I verified every layer with live tests:

- ☒ anon can write files to Storage (passes RLS)
- ☒ anon can insert `case_documents` (passes RLS)
- ☒ officer (authenticated) can read + View

The real cause: **every real upload you did happened before the policy fix**, so it got a 400, and **no new upload has happened since the fix** (live logs confirm — the latest POST was all yesterday, `case_documents` = 0 rows, Storage has no real customer file). With no files in the system, officers only see “Waiting”.

What made you think “it uploaded” is the app popping a green “uploaded” toast **before knowing the real result** — it lied.

What I just fixed and deployed (docs.js v=34)

- The toast shows ☒ **success only when both the upload and the record actually succeed.**
- On failure →  **with the real server error message** (not just a code), and the orphaned file is deleted so nothing looks done when it isn’t.

What to do now (1 step)

Wait ~1 min for deploy, then **upload one new file** in the app (hard refresh first): if it shows ☒ → open the officer side, that case’s Documents tab shows the file + a  View button immediately; if it shows  + **a message** → send me that message and I’ll fix the true root cause at once.

One limit: I can’t make real HTTP calls from here (the network policy blocks supabase.co), so your upload round is the final confirmation — but every layer behind it is proven to pass.

PROMPT 17

"The UI overlaps — fix it so status can be adjusted, and show the status like the other document types too."

RESPONSE

Done and pushed (Pages deploying ~1 min). Summary of the Documents-tab fixes on the officer side:

Root cause of the overlap: the `View` button used class `.btn` which sets `width:100%`, so it stretched across the row and covered the filename (which itself wrapped over ~4 lines).

Fixed (`staff.js v=8` , `staff.css v=2`): (1) View is now content-sized, moved onto its own actions row separate from the filename — no overlap; (2) long filenames truncate with `...` instead of growing the row; (3) added a **right-hand status badge** (Needs review / Verified ✓ / Re-request) like other rows, per your "show the status like the other document types"; (4) clicking a status updates the **badge instantly** (no reload).

After deploy, **hard refresh** the officer page — tell me if spacing/colors need more tuning.

PROMPT 18

"The layout in the red circle isn't nice — try to fit it into the yellow area; and don't forget, our app must be modern and suit first jobbers."

RESPONSE

Done and pushed (deploy ~1 min). Adjusted the layout as circled:

Fixed (`staff.css v=3`): the **View** button sits compactly on the left (40px tall) without taking too much room; the **segmented control** (Verified · Needs review · Re-request) moved **onto the same row as View** and stretches to fill the remaining yellow area — it no longer drops below; the 3 status buttons split the width evenly (`flex:1`) at the same height as View, reading as one clean, modern unit.

After deploy, **hard refresh** the officer page; if "Needs review" looks cramped on a very narrow screen, tell me and I'll shorten the label or tune the font.

PROMPT 19

"Make the back office look modern and beautiful, with colors aligned to the app, and a desktop-appropriate resolution."

RESPONSE

Done and pushed (deploy ~1 min). Turned the back office into a **real desktop console**, affecting only the officer app (screens $\geq 900\text{px}$), not the customer app or mobile:

Resolution / layout for desktop — from a 402px phone frame on black → a **full-height console, max 1280px**, centered; a **sticky full-width desktop header**; the **case queue + search as a card grid** (auto-fill, $\geq 330\text{px}$ cards) using the screen like a dashboard; **detail pages** (hub, docs, score, review, decide...) as a **centered 900px column**; the **login** as a centered card.

Colors aligned to the app — the same design tokens as the customer app (cobalt `#2563EB`, navy, lime, paper); a light-blue gradient desktop background (`--cobalt-50` → `--paper`); cards **hover** with a lifted shadow + soft cobalt border for a modern web-app feel.

Want me to extend it into a **left sidebar** (My queue / Search / profile) for a full back office? That looks even more professional but needs JS restructuring.

PROMPT 20

"Yes."

RESPONSE

Done and pushed (deploy ~1 min). Built the **full back-office sidebar**:

Left sidebar (desktop only, after login) — 🌱 logo + "Sprout · Officer console" at the top; **My tasks / Search cases** menu with an **active highlight** (the current one in cobalt); at the bottom an avatar + name + role + **Sign out**; on **navy per the brand** with white text — a premium, real-back-office feel.

Window structure — the whole app becomes a **floating console window, 24px rounded, shadowed**, centered (max 1320px); the right main area has a light-blue → white gradient, a sticky header, and content scrolling inside.

Mobile untouched — below 900px the sidebar hides and `.main-col` uses `display:contents` → a single-column flow exactly as before; duplicate role/logout in the top bar and the "signed in / search" row are hidden on desktop since the sidebar covers them.

PROMPT 21

“The links again, please.”

RESPONSE

App links (GitHub Pages — no custom domain, so the repo’s standard URLs):

Page	Link
 Home	https://kanychar-ai.github.io/Sprout-app/
 Customer app	https://kanychar-ai.github.io/Sprout-app/app/
 Back office (officer)	https://kanychar-ai.github.io/Sprout-app/staff/

Notes: open the **back office on desktop** to see the full sidebar — **hard refresh** (Ctrl/Cmd+Shift+R) the first time to load the new CSS/JS; Pages deploys several branches to the same URL and the latest push wins — currently branch `claude/lucid-volta-yrotb7` , just pushed, ~1 min to build. If it still looks old, tell me and I’ll check the build status.

PROMPT 22

“And where is the back office used to set up values?”

RESPONSE

Done and pushed (deploy ~1 min). Turned `/admin/` (**back office**) into a **modern desktop console, color-aligned to the app** — the same design as the officer console for system-wide consistency:

Desktop resolution + layout — a **floating, rounded, shadowed console window**, centered (max 1320px) on a light-blue gradient; a **navy brand sidebar**:  Loan products ·  Pre-screen rules ·  Documents ·  Staff & roles with an active highlight; below the sidebar, links to the Officer console / Customer app + the logged-in email + Sign out; **Products as a card grid** (dashboard-like) with hover lift.

Colors aligned — the same tokens (cobalt / navy / lime / paper) so every badge, button, switch and form matches the customer & officer apps; heading font Bricolage Grotesque like the main app.

No behavior change — CRUD over all 4 tables (products / prescreen_rules / doc_requirements / staff_roles) is identical; RLS ready (`authenticated` can write); on mobile the sidebar collapses to a top bar.

Link: <https://kanychar-ai.github.io/Sprout-app/admin/>

Note: it’s currently unrestricted — anyone who logs in via Supabase Auth can edit; I can gate it to role `admin` / `manager` in both UI and RLS if you want.

PROMPT 23

“For prescreening rules, please add specifying which product a rule belongs to via a dropdown. Imagine you’re an end user who finds things hard — the UI must be easy and quick, with Reset or Save.”

RESPONSE

Done and pushed (deploy ~1 min). Added **product-per-rule** with an easy UX, end-to-end:

Choose which product a rule is for — each rule card has a prominent **“Applies to” dropdown** (light-blue box) at the top: pick **“All products”** = applies to everything, or a **product name** (pulled from the products table automatically). Saved to a new column `prescreen_rules.product` (empty = all products).

Fast, easy Save / Reset — added a  **Reset** button (restore the saved values, discard pending edits) alongside  **Save rules**; the header + buttons are **sticky at the top** so you can Save/Reset anytime without scrolling back; the copy is short and clear: “tick → pick product → set values → Save”.

The customer app respects product scope — screening only uses/shows rules matching the customer’s chosen product (or rules set to All products). E.g. a “Minimum income” rule set only for “Auto Loan” won’t apply to other products.

Note: it’s currently **one rule per check type** (e.g. age has one row) with a product selector — if later you want the same check configured differently across products simultaneously, that needs multiple rows per check; say so and I’ll extend it.

PROMPT 24

"I want to pick a product first, then have the various rules as a subset of that product, so a user can choose to set up this product's rules."

RESPONSE

Done and pushed (deploy ~1 min). Switched to a **pick-product-first → set that product's rules** model, end-to-end:

New workflow (admin → Pre-screen rules) — (1) a **"Set up rules for"** dropdown at the top: **All products (default)** or a product name; (2) picking a product turns the checklist below into **that product's rule set** — toggle + configure freely; (3)  **Save** = create/update that product's rule set ·  **Reset** = discard pending edits (buttons sticky on top); (4) a hint shows the state: editing default / editing this product / this product has no set yet (it pre-fills from default so Save creates a new one).

Data model — `prescreen_rules` PK changed to `(product, key)` → separate rule sets per product; `product = ""` = the **default** set (the original 10 rules moved there automatically); unset products start from default and get their own set on Save.

The customer app respects it (with fallback) — a customer picks product A → if A has its own set, use it; otherwise use default; only **enabled** rules are used in the checklist and the pass/fail decision.

Now possible: Auto Loan age 20-60 + income ≥30k, Personal Loan age 25-55 + income ≥15k, separately; unset products still use default.

PROMPT 25

"How many % are left?"

RESPONSE

It depends what you count as "done" — I have no fixed pre-set plan, so here are two honest views from what's actually in the code:

As a demo/prototype that shows the full flow → ~90% — all 3 parts work and connect: the **customer app** (onboard → KYC → income → bank → docs uploaded to real Storage → tax → e-sign → prescreen → status → repay); the **officer console** (queue, review/approve maker-checker, view docs, set status — desktop + sidebar); the **back office** (manage products / per-product prescreen rules / documents / staff); DB + RLS + per-product rules ready.

As a real production lending system → ~60-65% — still stubbed/mock: e-KYC/OCR (format check only), the profile data used for prescreen (mock age 32/income 45k/score 720), the case customer name (hardcoded "Somchai Jaidee"), disbursement (simulated hand-off), admin permissions (unrestricted), real bureau/NCB and AML/sanctions (not connected).

Tell me the end goal (pitch/demo vs go-live) and I'll map the remaining % as a checklist. Which part should I tackle next? (I'd suggest using real form data in prescreen instead of the mock — highest impact on how "real" the system is.)

PROMPT 26

"Why so slow? Tell me what you're doing."

RESPONSE

Right now I'm **not running anything** — the last task (per-product rules) was committed + pushed, and I'm **waiting for your next instruction**; there's no background work.

What might feel slow: **GitHub Pages build** takes ~1 min after a push before the site updates (that's GitHub's side, not me working); the "how many %" answer earlier was just text, no code changes.

If you're waiting on something: (1) page not updating? → I can check the build status immediately; (2) want me to do something next? → just say it and I'll start. What would you like next?

PROMPT 27

"Remove All products from the dropdown; let the user set up one product at a time."

RESPONSE

Done and pushed (deploy ~1 min)  As requested — **removed "All products (default)" from the dropdown**. Now the dropdown shows **only product names**, defaults to the **first product** automatically → users set rules **one product at a time**; if there are no products yet, it shows "Add a product first, then set up its screening rules".

Behind the scenes: the default set still exists in the DB as a **template of check types** (age, income, ...) and as the **customer-app fallback** for products not yet configured — it just isn't shown in the dropdown; the customer side works exactly as before.

PROMPT 28

“Please order the customer-related data first — e.g. name, surname, ID, occupation — then the loan-request info, so staff instantly understand whose case and what it’s about, for maximum process efficiency, as if this staff member is the loan underwriter.”

RESPONSE

Done and pushed (deploy ~1 min)  Rebuilt the **Application data** page so the underwriter instantly sees “who / asking for what”:

1. Top summary (navy hero) — a glance says it all: left = who (avatar + name + ID + occupation), right = the ask (amount + product + term).

2. Applicant card first — leads with identity: Full name → National ID → Phone → Occupation → Employer → Monthly income → Existing debt.

3. Loan-request card after — Product → Amount → Term → Monthly → Purpose.

This order matches how a loan underwriter thinks: **confirm the customer’s identity → assess ability to pay → then consider the request**, cutting the time to figure out whose case it is.

PROMPT 29

“When the system checks the credit bureau, create a new database with ID and score so pre-screening can look up how many points a person has — e.g. Somchai Jaidee 1-1037-xxxxx-12-3 has 650.”

RESPONSE

Done and pushed (deploy ~1 min)  Built a real credit-bureau system:

New credit_bureau table (national_id → name, score) — RLS: the customer app (anon) can read, the back office (login) can edit; seeded with **Somchai Jaidee 1-1037-xxxxx-12-3 = 650**.

Pre-screening pulls the real score — on load it looks up the customer’s National ID in the bureau table → uses the real score in the “Minimum credit bureau score” check. E.g. Somchai 650 vs rule min 600 → **pass** (min 700 → fail); if lookup fails it falls back to 720.

New back-office tab:  **Credit bureau** — add/edit/delete a record: National ID + name + score → Save; staff manage each person’s score without touching Supabase.

Note: the customer’s National ID in the app is currently a fixed demo value (1-1037-xxxxx-12-3) because KYC is a stub — once real e-KYC reads the ID from the card, the system looks up each person’s real score automatically.

PROMPT 30

“Don’t repeat the headers every row, to save space, and allow adjusting the order — e.g. sort by ID number or first/last name — and split first and last name into separate fields.”

RESPONSE

Done and pushed (deploy ~1 min)  Adjusted the Credit-bureau table fully:

1. Headers once — space saved — `National ID · First name · Last name · Credit score` at the top; data rows below are pure inputs with no repeated labels → much more compact.

2. Split first / last name into 2 fields — added `first_name` + `last_name` columns (backfilled from the old name: Somchai / Jaidee); Save writes `first_name`, `last_name` and the combined name.

3. Sortable — a “Sort by” dropdown top-right sorts by **National ID / First name / Last name** (numeric-aware so IDs order correctly); editing a row and switching sort keeps typed values (synced to cache instantly).

PROMPT 31

"I just created this account — why doesn't Home show the name/surname I entered? And if income was never filled in, credit should be 0. When a customer registers, there should be a database that stores their data to display."

RESPONSE

Done and pushed (deploy ~1 min) ✓ Fixed all 3 points:

1. New `customers` table stores email (key), first_name, last_name, full_name, mobile, income. RLS: the customer app (anon) can create/update its own row; back office (staff) can read all.

2. Sign-up now saves the real name/surname/mobile — previously only email+password were sent; now it splits first/last, saves to localStorage + upserts to the DB, so Home shows the **real name entered** (with avatar initials), not "Somchai Jaidee".

3. Credit = ฿0 until income is entered — a freshly created account (income still 0) shows **฿0** + an "Add income to unlock your limit" badge; the limit is computed only once income > 0. Demo visitors are unaffected.

Honest notes: the income step doesn't yet write income back into `customers` (still a stub), so credit stays ฿0 for now; and login on a fresh browser (no localStorage) won't know the name yet — I can pull the profile back on login if you want.

PROMPT 32

"Yes. Is there anywhere else with mock-up data? Switch everything to values received from the user — no hardcoding."

RESPONSE

Done and pushed (deploy ~1 min) ✓ Overhauled so data comes from **real user input**, hardcoding removed:

New inputs added — KYC: National ID (13 digits), date of birth, gender. Income: Employer (was the dead string "SCG Co., Ltd."), real monthly income. Calculator: loan purpose.

The officer-facing case (submitCase) now: name (from sign-up), phone (entered mobile), National ID (from KYC), Employer (entered), income/debt (entered), purpose (chosen), **DSR = debt/income (computed)**, **Score/NCB computed** from bureau + affordability — every previously-hardcoded field is real.

Pre-screen (buildProfile) uses real values — age (from DOB), gender, occupation, pay type, income, computed DSR, and **credit score pulled from the bureau by the entered National ID; 0 if absent (not a fake 720)** → triggers real review.

Any screens that showed a fake name (role, recent activity, bank match, profile) now fill the real name/email. Income is written to `customers.income` at submit, so the Home limit reflects real income.

PROMPT 33

"Why does logging in with kanychar@hotmail.com still show Somchai on Home?"

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: the old login **never loaded a profile** — it only read the localStorage set at sign-up, so a login in a session without localStorage fell back to the HTML default "Somchai Jaidee".

Fixed: on successful login the system now (1) sets the profile to the logged-in email immediately (so "Somchai" is gone), and (2) fetches the `customers` row by email (authenticated read) to show the real name/mobile/income if present.

For kanychar@hotmail.com specifically: this account was created **before** the `customers` table existed and stored no name, so there's no name for it → after deploy it shows **"kanychar"** (from the email) + **฿0**, which is correct for the data on hand.

To give it a real name: sign up fresh with the name filled, or I can add a **profile-edit screen** so existing users can fill in their own name/mobile. Want the profile-edit screen?

PROMPT 34

“Change this page to Name and Surname and show what the customer typed — no hardcoding.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

The Create-account page now: splits into **two fields, Name + Surname**, side by side. Sign-up stores `first_name`, `last_name` and `full_name` from the real input, flowing everywhere — Home greeting, cards, the officer-facing case, and recovered on login. The **password-strength bar** no longer hardcodes “Strong” — it moves with what you type (length/digits/special chars): Weak → Fair → Good → Strong.

Next, to close the loop I recommend a **profile-edit screen** so existing users (e.g. kanychar, who has no name) can add their own — say the word.

PROMPT 35

“The surname runs off the screen — lay it out nicely for iPhone 17 Pro, and the green “S” at the top shouldn’t be there, remove it.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

- **Surname no longer overflows** — added `min-width:0` to both flex columns so Name/Surname shrink to a clean half-and-half, fitting iPhone 17 Pro.
- **Removed the green “S” chip** from the top bar (no JS referenced it — safe; the Me menu below still works).

PROMPT 36

“Pressing Back should go to the previous screen — verify this logic and fix it across the whole app.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Fixed Back to truly “return to the previous screen” app-wide:

Old problem: the stack started at `[home]` although the app opens at `onboard` → Back from the first step bounced to a home never seen; transient screens (role “signing in”, prescreen “checking”) were pushed → Back landed on un-returnable loading screens.

Fixed: (1) seed the stack from the actual opening screen; (2) separate **push (forward)** from **replace** — redirects role→home and prescreen→status use replace so Back **skips loading screens**; (3) browser/hardware Back stays consistent (pops instead of pushing when the hash goes to a prior screen).

Every screen with a back arrow (create, otp, calc, kyc, income, bank, docs, tax, status, products, me...) now steps back in the real order visited.

PROMPT 37

“This customer has no application in progress — design it as an empty state, and in the sign-up flow add steps so that if a customer fills in but doesn’t finish, Home shows which step they’re stuck on and tapping in resumes that step.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Built real application tracking + resume:

3 states of the “Your application” card — **Not started** (new customer): badge + “you have no application in progress...” + **Start application**; **In progress**: badge “X steps left” + “**you stopped at step N of 4** · [step name]” with progress dots + **Resume** · **Step N** opening exactly that step; **Submitted**: badge + “under review” + **Track status**.

Recent activity is real (not hardcoded) — a brand-new user sees only “Account created” (or “No activity yet”); in progress shows done steps (✓) + the current one (⌚ Next); submitted shows “Application submitted · under review”.

Mechanism: progress is saved to localStorage on entering each step (kyc→income→bank→docs), marked submitted at prescreen, cleared on a new application.

PROMPT 38

“Make this page look like the other image I uploaded, and on every page add the tab in the red circle so customers know how many steps remain; and when tapping the bank, let them choose (like the uploaded UI) with search-by-name.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Did all 3:

- 1. New Receiving-account screen (per mockup)** — from 4 bank buttons → a **single bank row** (logo + name + >) that opens a picker; the account number is a **real input** (was the dead “123-4-56789-0”); a “Name matches” card appears once a number is entered + a **yellow warning note** “must be your own name...”.
- 2. Select-bank screen (searchable)** — a “**Search bank...**” box filters by name/code instantly; 12 Thai banks with colored logos (KBank, SCB, BBL, KTB, Krungsri, ttb, GSB, BAAC, CIMB, UOB, KKP, LH); the chosen one shows ✓ and returns to update the row.
- 3. A 4-step progress bar on every flow screen** — kyc/income/bank/docs already had it; added **tax (“Almost done”)** and **e-sign (“Final step”)**, so every application step now shows progress.

PROMPT 39

“Hide what’s in the red circle, and test that the underwriting rule set for that product actually checks, and show in staff during underwriting why this customer didn’t pass auto pre-screening.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ All 3:

- 1. Removed the “Name matches” card** from Receiving account (only the yellow warning note remains).
 - 2. Confirmed the rules truly check** — inspected the DB config; keys match every evaluator and differ per product: **default** (any unset product) enables all 10 rules; **Loan not alone** disables age + paytype; **Salaryman Quick** disables nationality. The system pulls the chosen product’s rule set → compares to the real profile (age from DOB, income, computed DSR, credit score from bureau by National ID) → any violation = flag.
 - 3. Show the failure reasons on staff** — added `prescreen_pass` + `prescreen_fails` columns; at submit the system runs that product’s rules and records the result. The **officer case hub** shows a banner: green “Auto pre-screen: passed” if all pass, or red “Auto pre-screen: did not pass” + the **list of failed rules** (e.g. “Minimum credit bureau score”, “Minimum monthly income”) for that product; the Compliance menu shows a “Flagged” badge when failed.
- Example: apply with a National ID not in the bureau → `credit_score=0` → fails “Minimum 600” → the officer sees the reason immediately.

PROMPT 40

“Why is some data missing? Fix it.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Why data was missing: checking the DB, case SV578805 (Kany Char) had `national_id`, `occupation`, `employer` genuinely **empty** because the customer clicked Continue past those steps without filling them (they weren’t required); income/debt were unchanged defaults. Name/phone (from sign-up) were complete.

Fixed — require input before proceeding: **KYC** must have National ID, DOB, gender → otherwise a toast + can’t continue; **Income** must have occupation, employer → same. So cases reaching officers are complete.

Bonus: the officer Application-data page now shows “—” for blanks (easier to read for old incomplete cases).

Note: the old case (SV578805) stays incomplete (can’t backfill), but every new case will be complete — I can delete old test cases to tidy the DB if you like.

PROMPT 41

“This person just registered — why do they already have data? There must be none.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: application progress (`sprout_progress`) was stored in localStorage under a **single shared key not tied to an account** — so a browser that had once tested the flow to completion left “Submitted” behind, and a new account (kanych) saw the old data.

Fixed — bind progress to the account email: every save now **stamps the logged-in email**; on read, progress belonging to **another email (or old, email-less data)** is ignored → a new account sees “Not started” with no activity. Sign-up also resets the application state as a second layer.

Effect on kanych: simply **reload** (after deploy) and the old, un-stamped progress is ignored → back to “Not started”, \$0, no application. ✓

PROMPT 42

“Please give me the link into the admin backend.”

RESPONSE

Back-office admin link:

🖥️ <https://kanychar-ai.github.io/Sprout-app/admin/>

How to enter: log in with a staff account created in Supabase → Authentication (email + password); on **desktop** you’ll see the full sidebar (📁 Loan products · 🛡️ Pre-screen rules · 📄 Documents · 👤 Staff & roles · 📊 Credit bureau); **hard refresh** the first time.

Notes: logging in with a `staff_roles` role = `admin` auto-routes to admin even from the customer/officer entry; it’s currently unrestricted (anyone who can log in via Supabase Auth can edit) — I can lock it to admin/manager if you want.

Other links: 📱 Customer app /app/ · 🖥️ Officer console /staff/.

PROMPT 43

“I forgot the username/password.”

RESPONSE

✓ Password reset — you can log in now:

Credentials (both Officer and Admin): Email `approver@sprout.test` (or `reviewer@sprout.test`), Password `Sprout!2026`.

⚠️ Earlier slip: it was typed `approve@` — the correct one is `approver@sprout.test` (with the “r”).

Enter at: 🖥️ Officer console <https://kanychar-ai.github.io/Sprout-app/staff/> · ⚙️ Admin <https://kanychar-ai.github.io/Sprout-app/admin/> (log in with the same account).

Notes: both accounts are role **officer** (no admin/manager account yet), but `/admin/` is unrestricted so they can edit; I can set `approver@sprout.test` to `admin` to auto-route into admin; `Sprout!2026` is temporary and can be changed anytime.

PROMPT 44

“Opening on mobile, all the other tabs disappear.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: on mobile the nav became horizontal, but each button still had `width:100%` from desktop → the active button (“Loan products”) filled the row and pushed the other tabs off-screen.

Fixed: on mobile the tabs are **content-sized pills (width:auto) that wrap** onto a new line — all 5 visible: 📁 Loan products · 🛡️ Pre-screen rules · 📄 Documents · 👤 Staff & roles · 📊 Credit bureau.

PROMPT 45

“The links to open the staff and customer apps are gone, and make it like a mobile UI, but the desktop web should still be beautiful.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Fixed on the admin back office (mobile): the  **Officer console** +  **Customer app links are back** — shown as neat pills on the bottom row of the navy bar with Sign out on the right (they were hidden on mobile). The mobile layout is 3 clear rows: **brand → all tabs → app links + Sign out**. **Desktop is unchanged** — the full navy sidebar stays (only the mobile media query changed).

If you want the same cross-app links on the **Officer console** mobile view too, say so — the officer side doesn't have them yet.

PROMPT 46

“The UI isn't friendly and looks very cluttered — please adjust it.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Rearranged the mobile UI to be clean and app-like:

Mobile (before → after) — ✗ before: a tall navy block cramming everything (brand + tabs wrapping over 3 rows + links + signout), cluttered. ✓ after: a **slim top bar** (brand left · app links as **small round icons**   + Sign out right); the **tabs as a single horizontal scroll row** (swipe left-right), no wrapping, no clutter; the scrollbar hidden for a clean look.

Desktop unchanged — the full navy sidebar with text links ( Officer console) stays (only the mobile media query changed).

If anything still feels cramped (e.g. you'd prefer an iOS-style bottom tab bar instead of a scroll row), say so.

PROMPT 47

“The captured photos currently can't be seen by staff — let them be viewed in the customer-info part during underwriting.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: the e-KYC ID photos (front/back) were kept only on the customer's device, never uploaded to Storage → officers couldn't see them.

Fixed — end to end: (1) **customer side:** at submit the app uploads **front + back** to Supabase Storage (`caseId/id_front.jpg` , `id_back.jpg`) + records them in `case_documents` ; (2) **officer side (Application data / customer info):** a new “**ID document · e-KYC**” card under Applicant shows **front/back thumbnails**, tap to open full-size in a lightbox; if a case has none it shows “No ID photos were captured”.

Note: photos show for **new cases** captured after this deploy (old cases have none), and since KYC is a stub the image is the camera/chosen photo, not yet OCR-read from the card.

PROMPT 48

“Why can't I see the whole loan agreement?”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: the agreement box was locked to `130px; overflow:hidden` with a fade at the bottom (as a teaser) and the text was truncated with “...”, so it couldn't be read in full.

Fixed: the box is now **fully scrollable** (`max-height 240px; overflow-y:auto`) with the fade removed, and it contains the **full 8 clauses:** payment, EIR, early repayment, late fees, NCB reporting, transfer to a same-name account, PDPA, Thai law + e-signature.

Customers can read the whole agreement before accepting.

PROMPT 49

“How does the button in the red circle work? If it can’t actually work and isn’t useful, remove it.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

How the “Request another document” button worked: clicking it only **wrote a log** to the audit trail (“Requested another document”) + a toast “Requested · logged” — it **did not send any request to the customer** (the customer sees nothing, no document is specified, nothing new appears in the customer app) = a fake button with no real effect.

→ **Removed** as you said.

If you later want this feature to **actually work**, I can build it: officer taps → picks/types the needed document → creates a request tied to that case → the customer sees the extra document to upload in the app + a notification (needs a per-case document-request table). Just say so.

PROMPT 50

“Show already-disbursed but not-yet-repaid debt from every loan request; default to 0 if there is none.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Existing debt is now automatic from real debt — added `cases.applicant_email` to link each case to the customer account (saved at submit). The **Existing debt** field sums the **monthly of that customer’s disbursed cases** (disbursed but not closed). **No debt = ₦0** (new customers default to 0). The field is **read-only** (system-computed, grey with a “· auto” tag) and flows into DSR and pre-screen automatically.

Note: old cases created earlier have no `applicant_email` so they aren’t counted — only new cases after this deploy link to the account and add up.

PROMPT 51

“I disbursed 20,000 — why does it show 0?”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Switched to **National ID** as the key as requested:

Existing debt tied to the person (National ID) — sums the monthly of `disbursed` cases with a **matching national_id**; ₦0 if unknown. The National ID entered at KYC is **saved to the profile** so Calc can look it up next time (no re-entry).

For kanych’s case (SV570286): `national_id` `1103700539111`, monthly ₦3,491 — if the customer profile carries this ID, Calc shows Existing debt = **₦3,491**.

Important: since Calc comes before KYC, a **first-ever** application that hasn’t entered a National ID yet shows ₦0 (who is this?); once KYC is filled the ID is remembered and next time Calc pulls the debt automatically.

PROMPT 52

“It’s still 0.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Why still 0: Calc precedes KYC, so the profile had no National ID yet → the code found none = 0.

Fixed: if the National ID isn’t known, the system now **looks it up from this account’s cases** (matched by the account email/phone) → gets the `national_id` → remembers it → sums disbursed debt by that ID.

kanych’s case (SV570286) I’ve linked: `applicant_email` `kanychar@gmail.com`, `phone` `0640424246`, `national_id` `1103700539111`, monthly ₦3,491.

After deploy + **hard refresh**, if you’re logged in as the owner of this debt → Calc shows Existing debt = **₦3,491**.

⚠ If it’s still 0, the logged-in account doesn’t match what I linked — tell me the email you’re logged in with and I’ll link the debt case to it correctly (I guessed `kanychar@gmail.com`).

PROMPT 53

“Yes.”

RESPONSE

Great — confirming `kanychar@gmail.com` is correct means my link is right ✓

Everything’s ready: debt case SV570286 is linked to `kanychar@gmail.com` (national_id `1103700539111` , monthly ₦3,491); the new code (v=50) will, on login as kanychar, find the national_id from this case, sum disbursed debt, and show **₦3,491**.

To do: wait for deploy (~1 min) then **hard refresh** the customer app → open Calc → Existing debt shows **₦3,491** and DSR moves accordingly. If still 0 after a hard refresh: check you’re really logged in as kanychar@gmail.com (not demo mode), or tell me and I’ll check the logs/query.

PROMPT 54

“In real use the Capture and Use buttons should sit near the camera — they shouldn’t require sliding up and down. Adjust to user behavior.”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Adjusted to user behavior: moved the 📷 **Capture / Retake / Use photo** buttons **right under the camera frame** (next to the ✓ Good lighting status) — previously they were docked at the bottom of a long page, far from the camera, forcing a look-up-then-scroll-down.

New KYC order: camera → status line → [**📷 Capture**] **right there, near the camera** (Retake/Use on review) → Front/Back thumbnails → National ID/DOB/gender → PDPA note → [Continue] at the bottom. Your eyes stay on the camera and your thumb reaches the adjacent button — no sliding.

PROMPT 55

“Account kanychr@gmail.com already has a loan application, which means income was entered — why does Home show credit available = 0?”

RESPONSE

Done and pushed (deploy ~1 min) ✓

Cause: Home read income from `profile` (localStorage) = 0 after login (empty customers table), even though the real income lives on the **case** (cases.income); the “Your application” card read only localStorage so it showed “Not started” despite a real application in the DB.

Fixed — Home syncs from the real case: on opening Home it fetches the customer’s latest case (matched by National ID / email / phone) and (1) **uses that income to compute Available credit** (e.g. 45,000 → ₦180,000) and remembers it; (2) **shows the real status** on the card: In review / Approved / **Loan active** / Not approved, with **Track status** (or **Repay** if disbursed).

For kanychar (case SV570286 disbursed, income 45,000): Available credit shows **₦180,000** and the card shows **“Loan active”** + Repay.

PROMPT 56

“Please test that the rules set per product for prescreening actually check, and that every failing rule is recorded for staff.”

RESPONSE

✓ **Test results — confirmed per-product rules truly check, and all failures are recorded** (using exactly app.js’s logic + real DB rules):

Case	Product	Result	Note
Somchai (good)	Salaryman Quick	✓ PASS	9 rules checked (nationality off), all pass
Bad applicant	Salaryman Quick	✗ Under review	8 rules failed — all recorded
Age 17 + freelance	Loan not alone	✓ PASS	age & paytype off → not checked → pass
Same person	Salaryman Quick	✗ Under review	age & paytype on → 2 rules fail
Not in bureau	Nano/Micro (default)	✗ Under review	credit_score=0 < 600 → fail

4 things proven: (1) rules check per product — Salaryman Quick checks 9 (not nationality), Loan-not-alone checks 8 (not age+paytype), Nano/Micro has none → **falls back to default 10**; (2) same person, different product = different result; (3) failures record **every** rule (the bad applicant lists all 8 for staff, because `submitCase` uses `ev.fails.map(...)` and `prescreen_fails` is an array; the staff banner shows every line); (4) the bureau works — real scores by National ID (700/650) and 0 when absent → correctly fails.

Honest note: values are almost all real (age from DOB, gender, occupation, pay type, income, computed DSR, bureau credit) **except the documents rule**, still hardcoded `docsComplete=true` (always passes) since real upload isn’t checked yet — I can make documents check the real uploaded files if you want.

PROMPT 57

“Check documents for real too — required ones must be uploaded or it doesn’t pass.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Made the `documents` rule **check for real**:

Before: `docsComplete = true` fixed → the documents rule always passed (mock).

Now: `docs.js` adds `allRequiredUploaded()` — every document marked **Required** (except e-KYC auto ones like National ID) **must have a real uploaded file**; `buildProfile` feeds this into pre-screen → if a required doc is missing, the `documents` rule = **fail**; officers see the reason “All required documents uploaded” in the auto-prescreen banner.

Verified (same logic as app.js): documents complete → ✓ PASS; required missing → ✗ UNDER REVIEW · fails “All required documents uploaded”.

Pre-screen now checks every rule for real — age, gender, occupation, pay type, **documents (new)**, FATCA, credit bureau, income, DSR, nationality — all from real input (no mock left). This rule is toggleable per product in the back office like the others.

PROMPT 58

“Add a selfie step after taking the ID card, and show it on the staff back-end in the customer-info section.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Added a **selfie step** after the ID + shown on staff:

Customer side — new step: flow KYC (photograph card) → **Selfie** → Income. An “**Identity verification**” screen shows a mirrored front camera in a circle + a dashed ring + “● Detecting face...” + a **Capture selfie** button → Retake / Use photo; if the camera fails, a file fallback (capture="user"). The selfie is **required** before continuing; at submit it’s uploaded to Storage (`caseId/selfie.jpg`) + recorded in `case_documents` (doc_key `selfie`).

Staff side — Application data: an “**Identity · e-KYC**” card shows the **selfie as a round photo** + front/back thumbnails, each openable full-size. During underwriting the officer can compare selfie vs card.

Note: this stores the real camera image (no automatic face-match/liveness yet — “Detecting face...” is a visual cue); real face-matching needs an eKYC provider.

PROMPT 59

“Add a head frame showing where the face should be, tell the user to blink and turn left/right, auto-capture, then show the captured photo and let them preview it first.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Made the selfie a **guided liveness** flow:

Sequence (auto — no manual shutter): position your face in the circle → blink your eyes → slowly turn head left → turn head right → hold still → 📷 auto-capture. A **circle frame** shows where to place the face + a **progress ring** (blue) fills per step; when all steps complete it **captures automatically**, then enters **Preview** showing the shot with **Retake / Use photo**. Live view is mirrored (natural); the saved image is not (better for card comparison). No camera → a “Take a selfie photo” fallback.

A selfie is **required** before continuing; the photo is uploaded to Storage and shown on staff (round photo).

Honest note: the prompts are **scripted UX** (timed then auto-capture) — it doesn’t yet truly detect blinking/turning with ML; real liveness needs a face-detection library or eKYC provider.

PROMPT 60

“Why are the ID-card photo and some fields missing, and the selfie should show on this page too?”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Found the real cause in the DB:

1. ID photos + National ID + Phone missing — because this case (SV073392) **skipped the KYC step** (using “Resume” from Home jumped past it) → no card photo, no ID number. **Fixed:** added `identityErr()` — you must have an **ID number + both card photos + a selfie** (in this session) before continuing past income and before e-sign; if missing, you’re bounced back to **Verify identity** automatically → you can no longer submit while skipping KYC.

2. Selfie not showing — actually **the selfie is already in the DB** (this case has a `selfie` doc), but the staff page was a cached old version; “No ID photos were captured” was pre-v=15. **Fix:** just **hard-refresh** the Officer console → v=15 (which includes the selfie) loads → the round selfie appears in the “Identity · e-KYC” card.

Old case SV073392 stays incomplete (can’t backfill) but will show its selfie; new cases will be complete. Want me to delete old incomplete test cases to tidy up?

PROMPT 61

“Where did the phone go?”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Selfie + ID front/back now show nicely 🎉

Phone was missing because: it was read from `profile.mobile` (entered at sign-up) — this account logged in (empty customers table, no stored mobile), so it was blank.

Fixed — capture the number at KYC: added a **Mobile number** field on the identity card (after National ID), **prefilled** from the profile if present (recent sign-ups see their own number), **required** (part of the identity guard) so every application has a number, and written to the case → officers see Phone.

Identity now has every field: name, National ID, **Phone**, occupation, employer, income, debt + both ID photos + selfie.

PROMPT 62

“The phone number should be pulled from registration.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Confirmed the number is **pulled from registration** and made it clearer:

- The Mobile field is **auto-prefilled from the registered number** (seen in the screenshot = 0640424246).
- It’s now **locked (read-only)** when a registered number exists, with a “• from registration” tag next to the label so it’s clear where it came from.
- Accounts with no number on file (older logins) can still type one.

This number flows to the case so officers see Phone on every application.

If you’d rather let customers **edit** it here (in case of a number change) instead of locking, say so and I’ll unlock it.

PROMPT 63

“Add the logic below as an instruction — tell me what formula each number uses.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Made each score component have a **real formula** (computed from case data, summing to 100) + shown to officers.

Formulas (Affordability score 0-100): Repayment ability (DSR) /35 = $35 \times (1 - \text{DSR}/70)$ — low DSR = high score; **Credit bureau / NCB /30 = $30 \times (\text{bureau} - 300)/600$** — 300→0, 900→30, absent→0; **Income stability & tenure /20 = $\min(15, \text{income}/3000)$** + pay-type bonus (Payroll 5 / self-employed-or-owner 3 / other 1); **Existing obligations /10 = $10 \times (1 - \text{existing debt}/\text{income})$** ; **Past Sprout repayment /5 = new-customer baseline +2** (up to +5 with on-time history). **Total = sum of all 5** → 0-39 Low · 40-69 Fair · 70-100 Good.

Real example (Somchai: bureau 700, DSR 19%, income 45k, debt 8.5k, Payroll): DSR +26/35, Bureau +20/30, Income +20/20, Obligations +8/10, History +2/5 → **TOTAL 76** → **Good**.

How it works: the customer app computes at submit and stores `score_factors` (with formulas) on the case → the staff Score-breakdown page shows **+points/max + a formula line (f ...)** under each; old cases without factors fall back to computing from available fields.

PROMPT 64

“There’s an NCB score now — why doesn’t it show?”

RESPONSE

✓ Found the cause and fixed it fully:

Cause: the score was **computed and frozen at submit** — these cases were submitted **before** the bureau score existed in the table (or the cache at typing time hadn’t arrived) → bureau component = 0, ncb = “no-hit”. Adding the bureau later doesn’t update old cases by itself.

Fixed 2 ways: (1) Code (new cases): `submitCase` now **fetches the bureau score fresh at submit** (by the entered National ID) then computes → always uses the latest bureau data, no timing miss. **(2) Backfill (old case):** recomputed **SV671962 (Kanya Char, ID ...539888 = 650)** → score **67** → **84**, NCB **no-hit** → **clear**, bureau +17; hard-refresh the Officer page → the Score breakdown shows the real bureau points + total Good·84.

Test new cases with a National ID in the bureau (700/650/600/650) → the bureau score appears.

PROMPT 65

“Also show the date and time the app was submitted, sort oldest to newest, and color cases over 1 day differently so officers rush to process them.”

RESPONSE

Done and pushed (deploy ~1 min) ✓ Adjusted the task queue on all 3 points:

1. Show submit date/time — each case has a line `🕒 1 Jul, 14:02 · 3h ago` (real date/time + relative time). **2. Sort oldest→newest** — the queue sorts by `created_at ascending` (longest-waiting on top), FIFO. **3. Cases over 1 day = different color** — cases waiting > 24h get a **light-red background + red border** and an “🕒 **Over 1 day**” red tag. Hard-refresh the Officer console → My tasks → you’ll see submit times on every case and aged (>1 day) cases in red urging faster processing.

The threshold is 24h now — if you want a multi-level SLA (e.g. amber at 12h / red at 24h) or a different number of hours, say so.

PROMPT 66

“Even a Decline must show a reason, and there must still be an approver, and the approver must be able to forward the case to a manager.”

RESPONSE

Done — pushed to `claude/lucid-volta-yrotb7`. Summary of the 3 requested fixes:

1. A Decline always needs a reason — when the reviewer chooses “Recommend decline”, the note field clears its approve suggestion and shows a red “· reason required to decline” tag; submitting without a reason is blocked with a toast “A decline needs a reason”; the approver’s Reject still enforces a reason too.

2. There is an approver (maker-checker) — unchanged: the officer who reviewed a case can’t decide it (`reviewer_email === me.email` is guarded); the case goes to `pending_approval` for a different officer.

3. The approver can forward to a manager — the “Make decision” screen adds a **Send to a manager to decide** button that requires a note first, moving the case to `pending_manager`; escalation logic moved into the decision screen (replacing the old reasonless button) so a reason is always recorded; the manager’s view shows an “Escalated for a manager decision” card with the note.

Reason display — the case hub shows Reviewer note, Escalated-to-manager note, and Decision reason, and the audit log records every action. Added an `escalation_note` column to `cases` (migration applied) and bumped `staff.js?v=18`.

PROMPT 67

“Where are the decide button and the send-to-manager button?”

RESPONSE

Confirmed — case **SV671962** (status `pending_approval`) was reviewed by **reviewer@sprout.test (B)**, the account you’re logged in with, so the buttons are correctly hidden by separation of duties.

There are 3 accounts: **reviewer@sprout.test (B, officer)** — already reviewed → can’t decide (you’re on this one); **approver@sprout.test (A, officer)** — ☒ can **Make decision** and **Send to a manager**; **manager@sprout.test (M, manager)** — ☒ receives escalated cases for the final decision.

To see the buttons: (1) Sign out and log in as `approver@sprout.test` (password `Sprout!2026`); (2) open SV671962 → you’ll see **“Make decision · or send to a manager”**; (3) inside there’s ✓ Approve / × Reject and **Send to a manager to decide** (a note is required first); (4) if escalated → log in as `manager@sprout.test` for the final call.

One note: this case’s reviewer recommended `decline` but the note still says “Suggest approve \$30,000” — because it was reviewed **before** I enforced a decline reason (old data); new cases will require a consistent reason.